Object Oriented Programming (CSEG1044)

Unit 03 – Lecture 01: Nested Classes (Types + Use-cases)

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O
- **Course thread:** Use Nested Classes (Types + Use-cases) to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Class structure**: Nested classes still depend on the class-object model from Unit 02. Main reference: `Unit 02 / Lecture 01 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Class structure**: Nested classes still depend on the class-object model from Unit 02. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Interfaces and callbacks**: Anonymous and local classes make more sense once contract-based behavior is familiar. Main reference: `Unit 02 / Lecture 05 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain why nested classes improve encapsulation in some designs.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain why nested classes improve encapsulation in some designs.
- Differentiate static nested, inner, local, and anonymous classes.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain why nested classes improve encapsulation in some designs.
- Differentiate static nested, inner, local, and anonymous classes.
- Recognize use cases: helpers and callbacks/listeners (Swing later).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Terms to Know

- **Static nested class**: nested class with `static`; no implicit outer instance.
- **Inner class**: non-static member class; has implicit reference to outer object.
- **Local class**: class declared inside a method.
- **Anonymous class**: unnamed local class used inline to implement an interface/extend a class.
- **Callback**: passing code/behavior to run later (often via interfaces).
- **Effectively final**: a local variable that is not reassigned after initialization.
- **Outer instance**: the object of the outer class that an inner class "belongs to".

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).;

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Nested classes at a glance* → *Motivation: scope and encapsulation* → *Types of nested classes (with intuition)* → *Access rules (important and often surprising)*.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Nested classes at a glance* → *Motivation: scope and encapsulation* → *Types of nested classes (with intuition)* → *Access rules (important and often surprising)*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Nested classes at a glance* → *Motivation: scope and encapsulation* → *Types of nested classes (with intuition)* → *Access rules (important and often surprising)*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Unit 03: Nested Classes, Exceptions, Multithreading & IO Streams

- Lecture focus: Nested Classes (Types + Use-cases)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Nested class motivation: scope and encapsulation

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Nested class motivation: scope and encapsulation
- Static nested, inner, local, and anonymous classes

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Nested class motivation: scope and encapsulation
- Static nested, inner, local, and anonymous classes
- Use-cases: helpers, callbacks/listeners (bridge to Swing later)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Why nested classes?

- Keep helper types close to the code that uses them.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Why nested classes?

- Keep helper types close to the code that uses them.
- Reduce public API surface (better encapsulation).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Why nested classes?

- Keep helper types close to the code that uses them.
- Reduce public API surface (better encapsulation).
- Common for callbacks/listeners.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Types (quick)

- **Static nested**: no outer instance reference.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Types (quick)

- **Static nested**: no outer instance reference.
- **Inner**: has implicit outer instance reference.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Types (quick)

- **Static nested**: no outer instance reference.
- **Inner**: has implicit outer instance reference.
- **Local**: declared inside a method; limited scope.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Types (quick)

- **Static nested**: no outer instance reference.
- **Inner**: has implicit outer instance reference.
- **Local**: declared inside a method; limited scope.
- **Anonymous**: unnamed; used inline for one-time behavior.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Static nested vs inner (key difference)

- **Static nested**: behaves like a normal class scoped inside outer; no outer object needed.

Rule: if you do not need outer state, prefer `static`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Static nested vs inner (key difference)

- **Static nested**: behaves like a normal class scoped inside outer; no outer object needed.
- **Inner**: every instance is tied to an outer object (has an implicit outer reference).

Rule: if you do not need outer state, prefer `static`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: static nested helper (validator)

```
class BankAccount {
  static class Validator {
    static void requirePositive(double amt) {
      if (amt <= 0) throw new IllegalArgumentException("amt
          ");
    }
  }

  void deposit(double amt) {
    Validator.requirePositive(amt);
    // ... update balance
  }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: local class (inside a method)

```
public void demoLocal() {
  final int times = 2; // explicitly final

  class Printer {
    void say(String msg) {
      for (int i = 0; i < times; i++)
        System.out.println(msg);
    }
  }

  new Printer().say("Hi");
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Use cases

- Helpers: validators, builders, entries/tuples

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Use cases

- Helpers: validators, builders, entries/tuples
- Iterators and small adapters

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Use cases

- Helpers: validators, builders, entries/tuples
- Iterators and small adapters
- Callbacks/listeners (Swing event handling later)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Nested Classes (Types + Use-cases)* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Nested classes at a glance*, *Motivation: scope and encapsulation*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: inner + anonymous (small) (1/2)

```
interface Greeter { void greet(String name); }

public class Outer {
    private final String prefix = "Hello";

    class InnerGreeter implements Greeter {
        public void greet(String name) {
            System.out.println(prefix + ", " + name);
        }
    }
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: inner + anonymous (small) (2/2)

```
// continued...
    public void demo() {
        Greeter a = new InnerGreeter();
        Greeter b = new Greeter() { // anonymous
            public void greet(String name) {
                System.out.println(prefix + " (anon), " +
                    name);
            }
        };
        a.greet("UPES");
        b.greet("UPES");
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Which type has an implicit outer reference?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Which type has an implicit outer reference?
2. Which type is commonly used for callbacks?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Which type has an implicit outer reference?
2. Which type is commonly used for callbacks?
3. What does "effectively final" mean for local classes?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Confusing static nested classes with true inner classes.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Confusing static nested classes with true inner classes.
- Overusing anonymous classes when the logic is too large to read.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Confusing static nested classes with true inner classes.
- Overusing anonymous classes when the logic is too large to read.
- Scattering tiny helper types across many files without need.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Nested classes improve encapsulation and keep helpers local.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Nested classes improve encapsulation and keep helpers local.
- Know the 4 kinds: static nested, inner, local, anonymous.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Nested classes improve encapsulation and keep helpers local.
- Know the 4 kinds: static nested, inner, local, anonymous.
- Anonymous classes are common for callback-style code (Swing).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: static nested vs inner class?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: static nested vs inner class?
2. Why are anonymous classes used in event handling?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: static nested vs inner class?
2. Why are anonymous classes used in event handling?
3. Where can a local class be used?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Lecture Preview

Next: Exception Handling (Core)

- We will handle errors safely using `try/catch/finally` and custom exceptions.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).